

Reviewer Pack — Positive Eigenvalue Count Inverse Proportional to SOS Refuta...

Entry de5bbf7852f2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 11:18:00 UTC

Positive Eigenvalue Count Inverse Proportional to SOS Refutation Degree for Max-CUT

Entry ID: de5bbf7852f2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 11:18:00 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Refutation Degree for Max-CUT

Statement:

For a random Max-CUT instance on n variables, the number of positive eigenvalues of its degree- d moment matrix is $\geq \Omega(\log n / d^2)$. Conversely, if an instance's moment matrix has $\leq O(\log n / d^2)$ positive eigenvalues, then its SOS refutation degree exceeds d .

Rationale (proposer's reasoning):

The moment matrix's spectral structure encodes polynomial positivity constraints. Positive eigenvalues correspond to feasible directions in the SOS hierarchy, and their count provides a geometric obstruction to low-degree refutations. This bridges algebraic geometry's spectral analysis with SOS complexity.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0d2bc8a35c8d2536

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from typing import List, Dict

def gaussian_elimination(A: List[List[float]]) -> List[List[float]]:
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate non-pivot elements
        for j in range(n):
            if i != j:
                factor = A[j][i] / A[i][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]

    return A

def count_positive_eigenvalues(A: List[List[float]]) -> int:
    n = len(A)
    eigenvalues = [0] * n
    for i in range(n):
        if abs(A[i][i]) > 1e-9:
            eigenvalues[i] = A[i][i]
    return sum(1 for ev in eigenvalues if ev > 0)

def max_cut_instance(n: int) -> List[List[int]]:
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0
    return G

def degree_d_moment_matrix(G: List[List[int]], d: int) -> List[List[float]]:
    n = len(G)
    M = [[0.0 for _ in range(d+1)] for _ in range(d+1)]
```

```

    for i in range(n):
        for j in range(i, n):
            if G[i][j] == 1:
                for k in range(d+1):
                    for l in range(k+1):
                        M[l][k-l] += (i**l * j**(k-l) + j**l * i**(k-l)) / (2 * n)

    return M

def sos_refutation_degree(M: List[List[float]], d_max: int) -> int:
    for d in range(1, d_max+1):
        A = degree_d_moment_matrix(M, d)
        A = gaussian_elimination(A)
        if all(abs(A[i][i]) > 0 for i in range(d+1)):
            return d
    return d_max + 1

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)
    n = random.randint(5, 40)
    G = max_cut_instance(n)
    positive_eigenvalue_counts = []
    refutation_degrees = []

    for d in range(2, 9):
        M = degree_d_moment_matrix(G, d)
        eigenvalues = count_positive_eigenvalues(M)
        positive_eigenvalue_counts.append(eigenvalues)
        refutation_degree = sos_refutation_degree(M, d)
        refutation_degrees.append(refutation_degree)

    mean_positive_eigenvalues = sum(positive_eigenvalue_counts) / len(positive_eigenvalue_counts)
    max_refutation_degree = max(refutation_degrees)

    conjecture_holds = all(e >= math.log(n) / (d**2) for e, d in zip(positive_eigenvalue_counts, refutation_degrees))
    counterexample = "" if conjecture_holds else f"n={n}, refutation_degree={max_refutation_degree}"

    return {
        "metric_name": "mean_positive_eigenvalues",
        "metric_value": mean_positive_eigenvalues,
        "instances_tested": len(positive_eigenvalue_counts),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94717367.py", line 103, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94717367.py", line 80, in run_trial
    refutation_degree = sos_refutation_degree(M, d)
                        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94717367.py", line 64, in sos_refutation_degree
    A = gaussian_elimination(A)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_94717367.py", line 31, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError during Gaussian elimination, preventing result evaluation | next: Debug gaussian_elimination() for singular matrix handling

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110944
2	propose	ollama_remote	qwen3:8b	0	0	98675
3	propose	ollama_remote	qwen3:8b	0	0	114901

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	propose	ollama_remote	qwen3:8b	0	0	68275
5	preregistration	ollama_remote	qwen3:8b	0	0	25090
6	novelty	ollama_remote	qwen3:8b	0	0	20564
7	novelty	ollama_remote	qwen3:8b	0	0	14008
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18598
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12313
10	judge	ollama_remote	qwen3:8b	0	0	18040

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 501408 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/de5bbf7852f2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/de5bbf7852f2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/de5bbf7852f2.tar.gz` (if generated)